Expt.No:11 Date:

FREQUENCY RESPONSE OF FIR LOW PASS FILTER USING TRIANGULAR WINDOW

AIM:

To design an **FIR Low Pass Filter** using the **Triangular (Bartlett) Window** and analyze its frequency response using MATLAB.

TOOLS REQUIRED:

1) MATLAB Software,
2) Personal Computer.

THEORY:

A **Finite Impulse Response (FIR) filter** is a digital filter whose impulse response becomes zero after a finite number of samples. FIR filters are widely used in digital signal processing because they are **always stable**, can provide an **exact linear phase response**, and are easy to implement.

The output of an FIR filter is given by

$$y(n) = \sum_{k=0}^{N-1} h(k)x(n-k)$$

where:

- $x(n)$= Input signal
- $y(n)$= Output signal
- $h(k)$= Filter coefficients
- N= Filter length

An **FIR Low Pass Filter** passes low-frequency components while attenuating high-frequency components above the cutoff frequency.

Window Method for FIR Filter Design

The impulse response of an ideal low-pass filter is infinite in length and cannot be implemented directly. Therefore, it is multiplied by a finite-duration **window function** to obtain a realizable FIR filter.

The window method is represented as

$$h(n) = h_d(n) \times w(n)$$

where:

- $h_d(n)$= Ideal impulse response
- $w(n)$= Window function
- $h(n)$= FIR filter coefficients

The choice of the window determines the filter's transition width and stopband attenuation.

Triangular (Bartlett) Window

The **Triangular Window**, also known as the **Bartlett Window**, is a window function whose amplitude increases linearly to the center and then decreases linearly toward the edges, giving it a triangular shape.

The Bartlett window is mathematically defined as

$$w(n) = \begin{cases} \dfrac{2n}{N-1}, & 0 \le n \le \dfrac{N-1}{2} \\ 2 - \dfrac{2n}{N-1}, & \dfrac{N-1}{2} \le n \le N-1 \end{cases}$$

where:

- N= Length of the window
- $0 \le n \le N-1$

PROCEDURE:

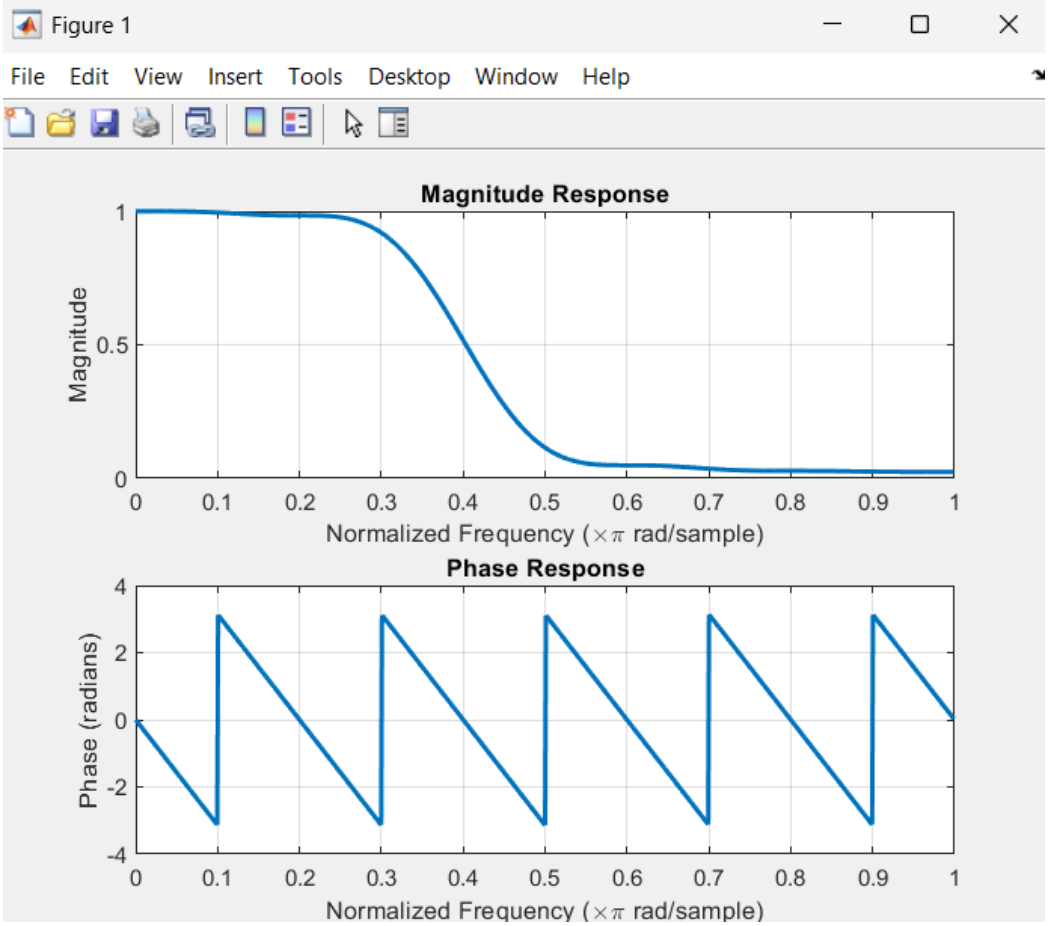
1. Open MATLAB.
2. Click Home → New Script.
3. Enter the MATLAB program given below.
4. Save the file as fir_triangular_window.m.
5. Click Run.
6. MATLAB designs an FIR Low Pass Filter using the Bartlett (Triangular) window.
7. Observe the magnitude and phase responses displayed by MATLAB.
8. Analyze the passband, transition band, and stopband characteristics.

MATLAB PROGRAM

```
clc;
clear;
close all;
N = 20;
fc = 0.4;
w = bartlett(N+1);
b = fir1(N,fc,'low',w);
[H,f] = freqz(b,1,1024);
figure;
subplot(2,1,1);
plot(f/pi,abs(H),'LineWidth',1.5);
xlabel('Normalized Frequency (\times\pi rad/sample)');
ylabel('Magnitude');
title('Magnitude Response');
grid on;
```

```
subplot(2,1,2);
plot(f/pi,angle(H),'LineWidth',1.5);
xlabel('Normalized Frequency (\times\pi rad/sample)');
ylabel('Phase (radians)');
title('Phase Response');
grid on;
```

MODEL GRAPH:



Result

Thus, the FIR Low Pass Filter was successfully designed using the Triangular (Bartlett) Window and its frequency response was obtained.

Expt.No:12 Date:

VOICE ACTIVITY DETECTION (VAD) IN SPEECH

AIM:

To detect the voice (speech) and silence (non-speech) regions of a speech signal using Voice Activity Detection (VAD) in MATLAB.

TOOLS REQUIRED:

1) MATLAB Software,
2) Personal Computer.

THEORY

Voice Activity Detection (VAD) is a technique used in Digital Signal Processing (DSP) and Speech Processing to identify the portions of an audio signal that contain human speech and distinguish them from silence or background noise. The main objective of VAD is to determine whether speech is present in a given audio frame.

VAD is widely used in speech communication systems to reduce unnecessary processing, improve transmission efficiency, and enhance speech recognition performance.

The speech signal is divided into short-duration frames (typically 10–30 ms). For each frame, specific features such as short-time energy, zero-crossing rate, or spectral characteristics are computed. These features are then compared with predefined thresholds to classify the frame as either speech or non-speech.

Voice Activity Detection (VAD)

Voice Activity Detection is a binary classification process in which each frame of a speech signal is labeled as:

- Voice (Speech)
- Silence (Non-Speech)

A VAD system generally consists of the following stages:

1. Speech signal acquisition.
2. Framing and windowing.
3. Feature extraction.
4. Threshold comparison.
5. Voice activity decision.

Short-Time Energy (STE)

The Short-Time Energy (STE) is one of the simplest and most widely used features for detecting speech activity. Speech frames usually have higher energy than silence frames.

The short-time energy for a frame is calculated as

$$E = \sum_{n=0}^{N-1} | x(n) |^2$$

where:

- $x(n)$= Speech samples
- N= Number of samples in a frame
- E= Short-time energy

Frames with energy greater than a threshold are classified as speech.

Zero Crossing Rate (ZCR)

The Zero Crossing Rate measures how frequently the speech signal changes sign within a frame. It is defined as

$$ZCR = \frac{1}{2N} \sum_{n=1}^{N-1} | \ \text{sgn}(x(n)) - \text{sgn}(x(n-1)) \ |$$

where:

$$\text{sgn}(x) = \begin{cases} 1, & x \geq 0 \\ -1, & x < 0 \end{cases}$$

Speech and silence generally have different ZCR characteristics, making it useful for voice detection.

PROCEDURE:

1. Open MATLAB.
2. Click Home → New Script.
3. Enter the MATLAB program.
4. Save the file as vad.m.
5. Place the speech file (e.g., speech.wav) in the current working directory.
6. Run the program.
7. The speech signal is read and divided into short frames.
8. The short-time energy of each frame is calculated.
9. A threshold is computed based on the maximum frame energy.
10. Frames with energy above the threshold are classified as speech.
11. Observe the waveform, frame energy, and detected voice activity.

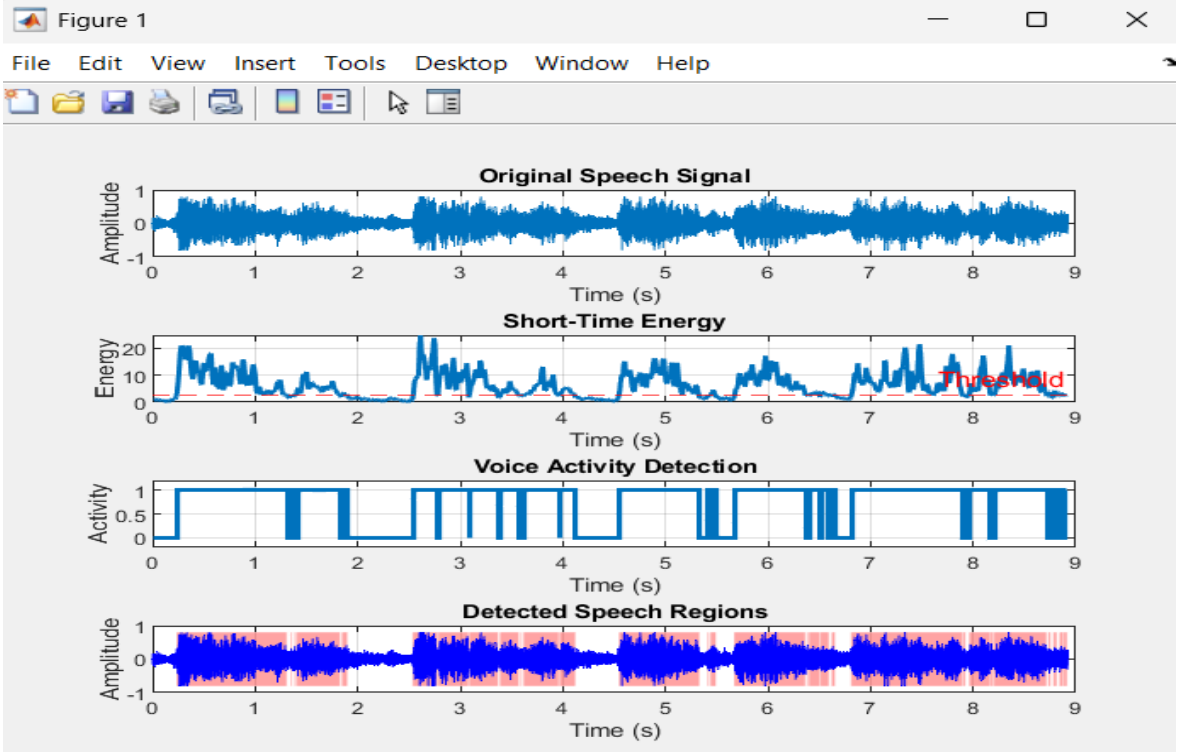
MATLAB PROGRAM

```
clc;
clear;
close all;
load handel
audiowrite('speech.wav',y,Fs)
%% Read Speech Signal
[x, Fs] = audioread('speech.wav');
% Convert stereo to mono if necessary
if size(x,2) == 2
    x = mean(x,2);
end
%% Time Axis
t = (0:length(x)-1)/Fs;
%% Frame Blocking
frameLength = round(0.02*Fs);    % 20 ms
frameShift  = round(0.01*Fs);    % 10 ms overlap
numFrames = floor((length(x)-frameLength)/frameShift) + 1;
%% Compute Short-Time Energy
energy = zeros(numFrames,1);
for k = 1:numFrames
    startIndex = (k-1)*frameShift + 1;
    frame = x(startIndex:startIndex+frameLength-1);
    energy(k) = sum(frame.^2);
end
%% Threshold Selection
threshold = 0.1*max(energy);
%% Voice Activity Detection
vad = energy > threshold;
%% Frame Time Axis
frameTime = ((0:numFrames-1)*frameShift + frameLength/2)/Fs;
%% Plot Results
figure;
% Original Speech Signal
subplot(4,1,1);
plot(t,x);
```

```
xlabel('Time (s)');
ylabel('Amplitude');
title('Original Speech Signal');
grid on;
% Short-Time Energy
subplot(4,1,2);
plot(frameTime,energy,'LineWidth',1.5);
hold on;
yline(threshold,'r--','Threshold');
xlabel('Time (s)');
ylabel('Energy');
title('Short-Time Energy');
grid on;
% VAD Output
subplot(4,1,3);
stairs(frameTime,vad,'LineWidth',2);
ylim([-0.2 1.2]);
xlabel('Time (s)');
ylabel('Activity');
title('Voice Activity Detection');
grid on;
% Highlight Detected Speech Regions
subplot(4,1,4);
plot(t,x,'b');
hold on;
for k = 1:numFrames
   if vad(k)
      startTime = (k-1)*frameShift/Fs;
      endTime = startTime + frameLength/Fs;
      patch([startTime endTime endTime startTime], ...
          [min(x) min(x) max(x) max(x)], ...
          'r', ...
          'FaceAlpha',0.2, ...
          'EdgeColor','none');
   end
end
```

```
plot(t,x,'b');
xlabel('Time (s)');
ylabel('Amplitude');
title('Detected Speech Regions');
grid on;
%% Display Information
fprintf('Sampling Frequency = %d Hz\n', Fs);
fprintf('Frame Length       = %d samples\n', frameLength);
fprintf('Number of Frames   = %d\n', numFrames);
fprintf('Energy Threshold   = %.6f\n', threshold);
```

MODEL GRAPH:



COMMAND WINDOW:

Sampling Frequency =

Frame Length =

Number of Frames =

Energy Threshold =

RESULT

Thus, Voice Activity Detection was successfully implemented using short-time energy and speech/non-speech regions were identified.